

Contenedores y ejecución reproducible con Docker

Del entrenamiento local a una ejecución portable, verificable y delimitada.

IMAGEN

qué entorno de software queda fijado

CONTRATO

qué condiciones declara la ejecución

EVIDENCIA

qué permite contrastar dos hosts

Que funcione en una máquina no identifica sus condiciones de ejecución



PREGUNTA DE AUDITORÍA

¿Qué condición cambió y dónde debía quedar declarada o registrada?

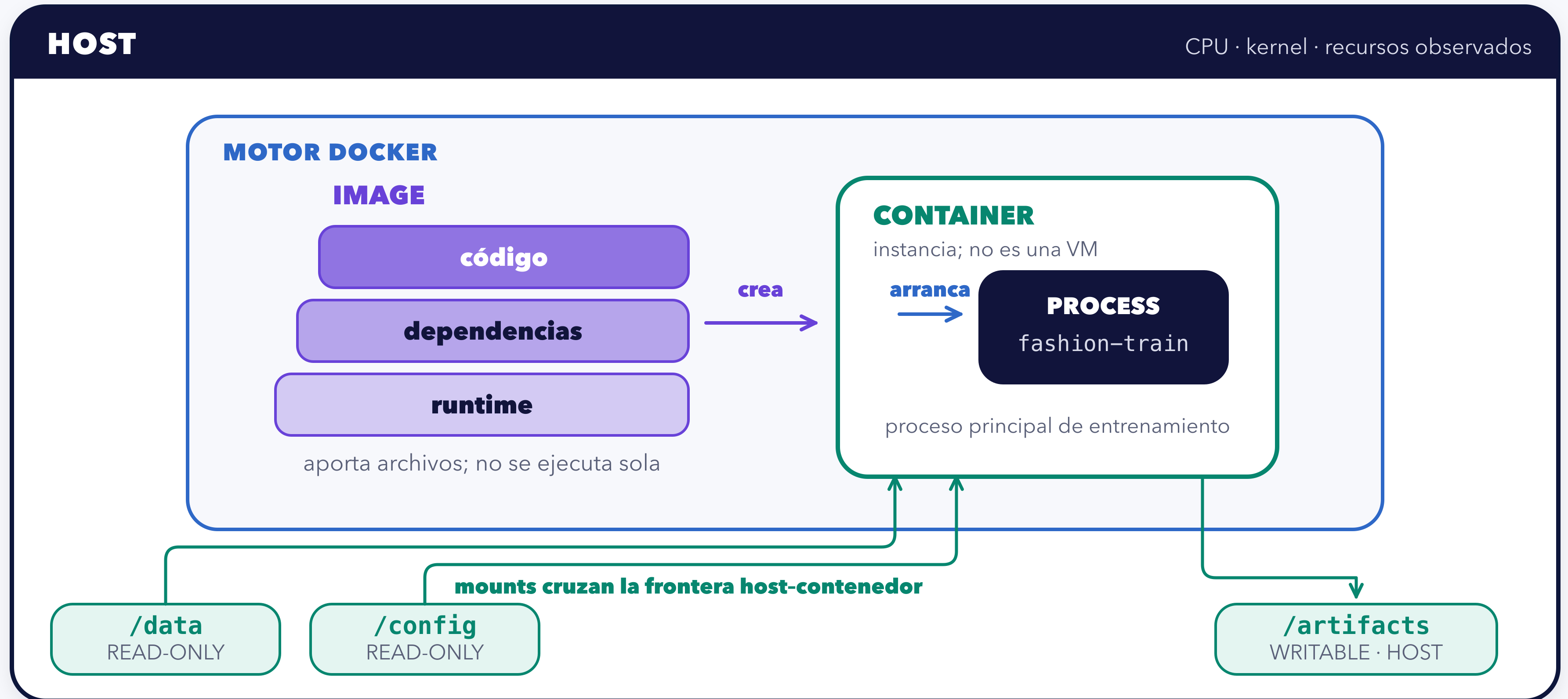
Cada discrepancia exige nombrar el mecanismo que puede controlarla

FUENTE DE VARIACIÓN	BUILD fija en la imagen	RUN declara al ejecutar	HOST / CÁLCULO se observa o limita
CÓDIGO	B COPY + contexto		
RUNTIME	B imagen base		
DEPENDENCIAS	B lock + instalación		
DATOS		R bind mount R0	
CONFIGURACIÓN		R archivo + CLI	
HARDWARE			H CPU / kernel / arquitectura
ARITMÉTICA			H plataforma / release
RESULTADOS		R bind mount RW	

Cada asignación debe nombrar un mecanismo verificable.

La imagen no controla hardware ni aritmética.

La imagen aporta archivos; el contenedor ejecuta un proceso sobre un host



Eliminar el contenedor no elimina la imagen ni los archivos externalizados

EVENTO	IMAGE	CONTAINER instancia + capa	BIND MOUNT archivos en host
PROCESO TERMINA	✓ permanece	✓ detenido + capa	✓ archivos permanecen
docker rm	✓ permanece	✗ se eliminan	✓ archivos permanecen
NUEVO docker run	✓ se reutiliza	+ instancia nueva	+ se remonta
docker rmi*	✗ se elimina	✓ sin cambio	✓ sin cambio
* si la imagen no está en uso			

PREDICCIÓN PARA EL CONTRATO DEL CASO

/app/model-state.pt

BLOQUEADO

raíz read-only

/tmp/model-state.pt

TEMPORAL

tmpfs se pierde

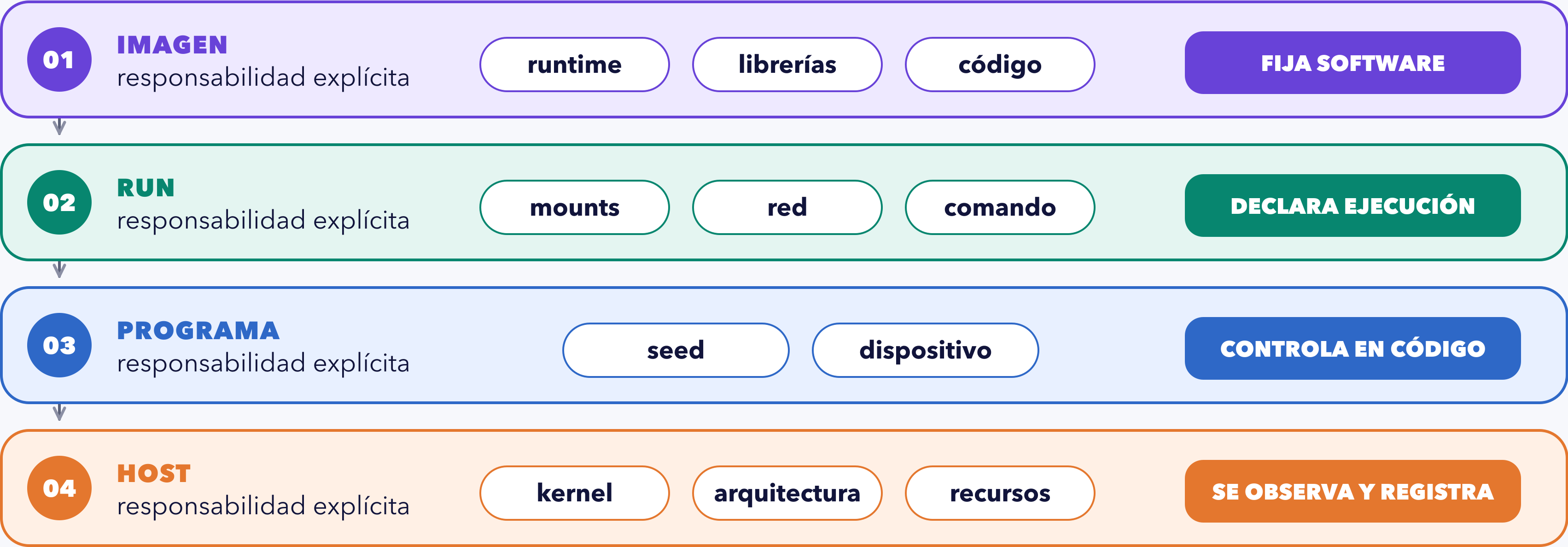
/artifacts/model-state.pt

PERSISTE

archivo en host

La misma imagen reduce variación de software, pero no fija datos, hardware ni aritmética

Cuatro niveles complementarios delimitan qué se fija, qué se declara y qué todavía debe observarse.



AFIRMACIÓN ACOTADA

Una imagen común reduce variación de software; no garantiza igualdad numérica entre hosts.

En docker run, las opciones preceden a la imagen y los argumentos del programa la siguen

La posición separa la autoridad de Docker de la autoridad de la aplicación.

GRAMÁTICA • UN SOLO BASELINE

docker run

[OPCIONES]

IMAGEN

[ARGUMENTOS]

1 OPCIONES DE DOCKER

Configuran el contenedor:

`mounts · red · escritura`

Deben aparecer antes de la imagen.

2 IDENTIDAD DE IMAGEN

Marca la frontera posicional.

`fashion-train:local`

Selecciona el paquete ejecutable.

3 CLI FASHION-TRAIN

Configura el proceso:

`datos · config · salida`

Debe aparecer después de la imagen.

ANTES DE IMAGEN = Docker

|

DESPUÉS DE IMAGEN = fashion-train

El comando declara limpieza, red, escritura, entradas, salida y programa

Cada fragmento controla una condición observable; ninguno reemplaza el registro de la imagen y los datos.

COMANDO DE REFERENCIA • HOST POSIX

pendiente de preflight

```
docker run --rm \
--pull=never \
--network none \
--read-only \
--tmpfs /tmp:rw,noexec,nosuid,size=64m \
--mount type=bind,src="$(pwd)/data",dst=/data,readonly \
--mount type=bind,src="$(pwd)/config",dst=/config,readonly \
--mount type=bind,src="$(pwd)/artifacts/run-01",dst=/artifacts \
fashion-train:local \
--data-root /data \
--split-manifest /config/split-indices.json \
--config /config/train-config.json \
--output-dir /artifacts
```

CONDICIÓN CONTROLADA

LIMPIEZA

--rm descarta el contenedor;
--pull=never evita descarga implícita.

RED

--network none conserva solo
la interfaz loopback.

ESCRITURA

raíz read-only; /tmp temporal
y sin persistencia.

ENTRADAS

/data y /config se montan
como solo lectura.

SALIDA

/artifacts externaliza
resultados necesarios.

IMAGEN

fashion-train:local selecciona
la imagen local.

ARGUMENTOS

los argumentos finales configuran
el proceso fashion-train.

[OPCIONES DEL CONTENEDOR] → IMAGEN → [ARGUMENTOS DEL PROGRAMA]

El comando formula condiciones observables; el ledger deberá registrar la imagen y las entradas ejecutadas.

Un contrato de ejecución es útil cuando permite predecir también sus fallos

Cada opción debe producir un comportamiento observable y causalmente explicable.

CONDICIÓN	OPCIÓN RESPONSABLE	PREDICCIÓN OBSERVABLE
source del bind mount no existe	<code>--mount type=bind</code>	→  FALLO ESPERADO Docker rechaza el mount
Fashion-MNIST intenta descargarse	<code>--network none</code>	→  FALLO ESPERADO no hay red externa
el proceso intenta modificar /data	<code>readonly en /data</code>	→  FALLO ESPERADO la entrada está protegida
escritura fuera de /tmp o /artifacts	<code>--read-only raíz del contenedor</code>	→  FALLO ESPERADO la raíz rechaza escritura
el proceso termina correctamente	<code>--rm + mount /artifacts</code>	→  ÉXITO ESPERADO contenedor fuera; salida persiste

Validar el contrato significa contrastar estas predicciones; no basta con leer que las opciones están presentes.

La arquitectura se inspecciona para fijar entradas y salidas, no para rediseñarla

El objetivo es verificar el contrato de formas del MLP entregado, no elegir otra red.

ARQUITECTURA ENTREGADA

NO REDISEÑAR

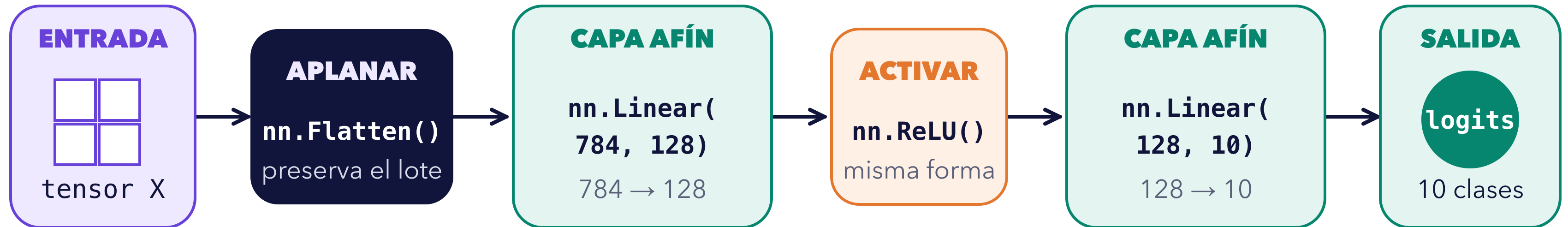
inspección de entrada → salida

$X : (B, 1, 28, 28)$

$(B, 784)$

$(B, 128)$

$\hat{y} : (B, 10)$



CONTRATO QUE SE VERIFICA

La entrada y la salida coinciden con el programa; no se añaden capas ni métricas.

Seed, CPU, orden de datos y download=False pertenecen al contrato del programa

La imagen fija software; estos controles todavía deben declararse dentro de `fashion-train`.

CONTROL EN CÓDIGO	VARIACIÓN QUE REDUCE	LÍMITE QUE PERMANECE
<code>configure_reproducibility(seed)</code>	→ seed de Python y PyTorch	release y plataforma pueden diferir
<code>torch.Generator().manual_seed(seed)</code>	→ orden del shuffle	algoritmos disponibles siguen importando
<code>DataLoader(..., num_workers=0)</code>	→ paralelismo del loader	reduce concurrencia; no prueba equivalencia
<code>FashionMNIST(..., download=False)</code>	→ descarga durante el run	el dataset correcto debe existir

≠ Reducir variación no equivale a identidad absoluta entre releases, plataformas o dispositivos.

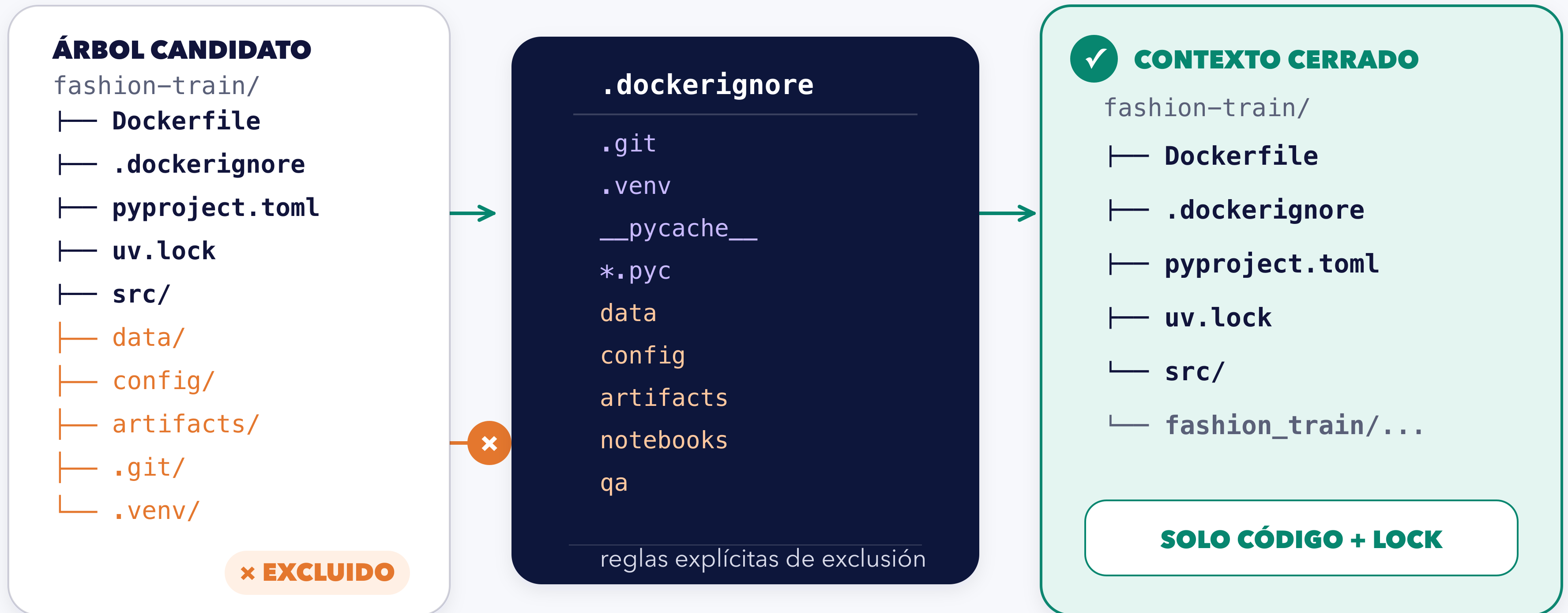
Datos y configuración entran sin escritura; la evidencia sale por un directorio distinto

Cada archivo pertenece a una fase y a una regla de mutabilidad explícitas.



El contexto de build contiene código y lock, no datos, artefactos ni estado local

El filtro limita lo que el Dockerfile puede copiar y hace visible el estado excluido.



El Dockerfile solo puede copiar lo que atraviesa el contexto de build.

La base, el lock y la plataforma deben verificarse antes de congelar la imagen

Un candidato no se vuelve evidencia hasta resolver los valores para Linux CPU y la plataforma objetivo.



VERIFICADO

0 valores congelados: el preflight todavía no aporta evidencia observada.

CANDIDATOS • NO CONGELAR

BASE PROPUESTA

`python:3.12-slim`

PLATAFORMAS POR RESOLVER

`linux/amd64 · linux/arm64`
o una sola plataforma soportada

Estado: hipótesis de trabajo, no certificación.

BLOQUEANTES • EVIDENCIA REQUERIDA



PyTorch + torchvision

combinación Linux CPU compatible



digest de la base

digest real por plataforma



UV_VERSION

versión verificada



uv.lock

lock resuelto para Linux CPU

BUILD CERRADO

Los placeholders y el lock macOS no se presentan como una imagen Linux ejecutable.

Las capas separan runtime, dependencias y código para hacer visible qué cambia

El orden permite reutilizar trabajo; un cache hit no certifica reproducibilidad.

Dockerfile · CANDIDATO

```
ARG PYTHON_BASE=python:3.12-slim@sha256:<DIGEST_VERIFICADO>
```

```
FROM ${PYTHON_BASE}
```

```
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app/src
```

```
RUN pip install --no-cache-dir uv==${UV_VERSION}
```

```
COPY pyproject.toml uv.lock ./
```

```
RUN uv sync --frozen --no-dev --no-install-project
```

```
COPY src ./src
```

```
ENTRYPOINT ["python", "-m", "fashion_train.train"]
```

Los dos placeholders bloquean el build hasta el preflight.

1 · RUNTIME

Base y entorno del proceso.

2 · DEPENDENCIAS

Instala uv, copia lock y sincroniza.
Cambiar el lock invalida esta etapa.

3 · CÓDIGO

Se copia después de resolver dependencias.

4 · PROCESO PRINCIPAL

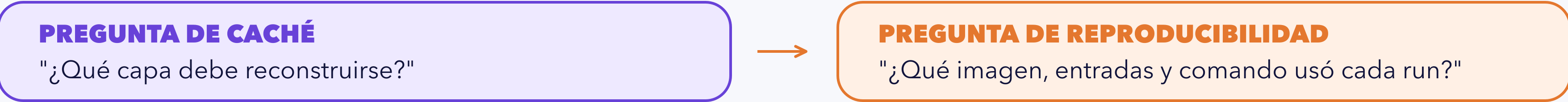
ENTRYPOINT delega en fashion-train.

La caché responde qué se reconstruye; no responde si dos runs son equivalentes.

La caché acelera el build; no demuestra que dos ejecuciones usaron el mismo contrato

Predecir invalidaciones explica el build; comparar runs exige otra evidencia.

CAMBIO	CAPA AFECTADA	REINSTALA	POR QUÉ
src/fashion_train/train.py	COPY src	NO	el código llega después de uv sync
uv.lock	COPY lock → uv sync	SÍ	cambia la entrada de dependencias
data/FashionMNIST/...	fuera del contexto	NO	los datos entran durante el run
.dockerignore	contexto enviado	DEPENDE	puede cambiar qué alcanza el build



Una etiqueta local no sustituye el image ID ni la plataforma observada

El build se acepta cuando el comando, el código de salida y la identidad observada quedan registrados.

1 • BUILD

```
docker build --tag fashion-train:local .
```

Solo cuando el preflight esté resuelto.

2 • INSPECT

```
docker image inspect \  
--format \  
'{{.Id}} {{.Os}}/{{.Architecture}}' \  
fashion-train:local
```

La salida real se copia al registro; aquí no se anticipa.

REGISTRO DE EVIDENCIA OBSERVADA

Los campos quedan vacíos hasta ejecutar build e inspect.

exit code

POR OBSERVAR

image ID

POR OBSERVAR

OS/arquitectura

POR OBSERVAR

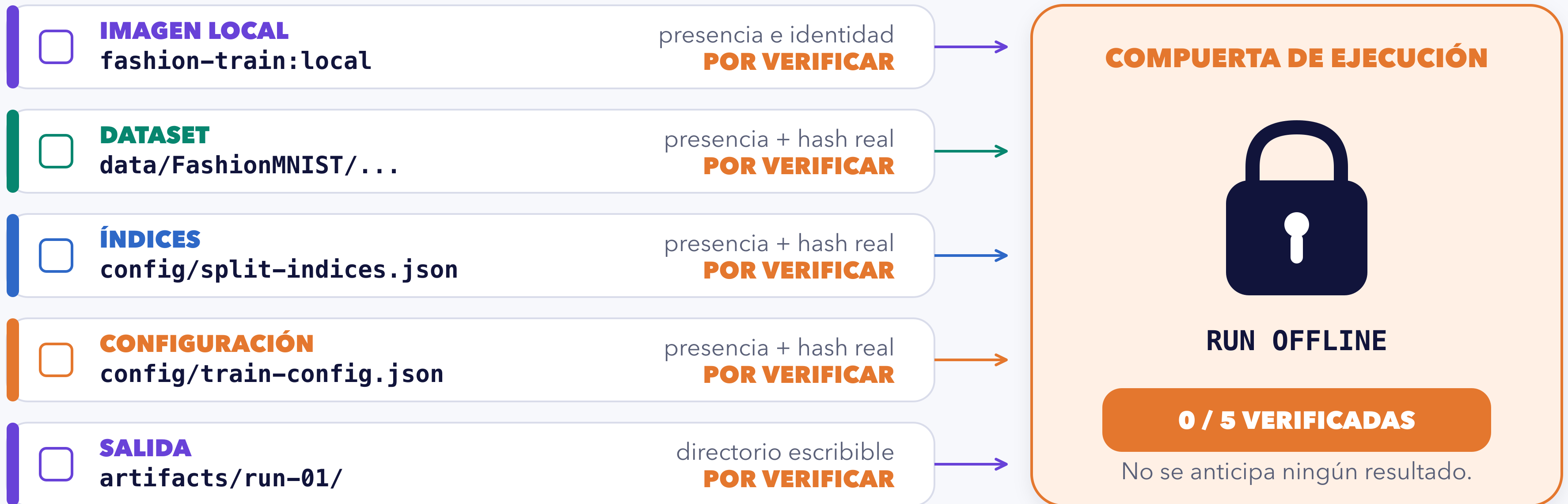
ACEPTAR SOLO CON LOS TRES CAMPOS

fashion-train:local es una etiqueta mutable; no reemplaza el image ID ni la plataforma.

La ejecución verificada comienza con imagen y entradas ya disponibles

El run offline no descarga ni improvisa: comprueba presencia, hashes y permisos en el host.

PREFLIGHT DEL HOST • 5 CONDICIONES



SEPARAR PREPARACIÓN Y ENTRENAMIENTO

Dataset preparado fuera del contenedor → `download=False` + `--network none` durante el run verificado.

El run aplica exactamente el contrato de red, escritura, entradas, salida y programa

El comando sigue siendo referencia hasta capturar evidencia real de la ejecución.

COMANDO DE REFERENCIA • HOST POSIX

```
docker run --rm \
--pull=never \
--network none \
--read-only \
--tmpfs /tmp:rw,noexec,nosuid,size=64m \
--mount type=bind,src="$(pwd)/data",dst=/data,readonly \
--mount type=bind,src="$(pwd)/config",dst=/config,readonly \
--mount type=bind,src="$(pwd)/artifacts/run-01",dst=/artifacts \
fashion-train:local \
--data-root /data \
--split-manifest /config/split-indices.json \
--config /config/train-config.json \
--output-dir /artifacts
```

no ejecutado en esta evidencia

LECTURA FUNCIONAL

LIMPIEZA

--rm · --pull=never

RED

--network none

ESCRITURA

raíz R0 · /tmp temporal

ENTRADAS

/data · /config · R0

SALIDA

/artifacts · RW

PROGRAMA

imagen + argumentos

LEDGER INICIAL • SE COMPLETA DESPUÉS DEL RUN

COMANDO EXACTO host
POR OBSERVAR

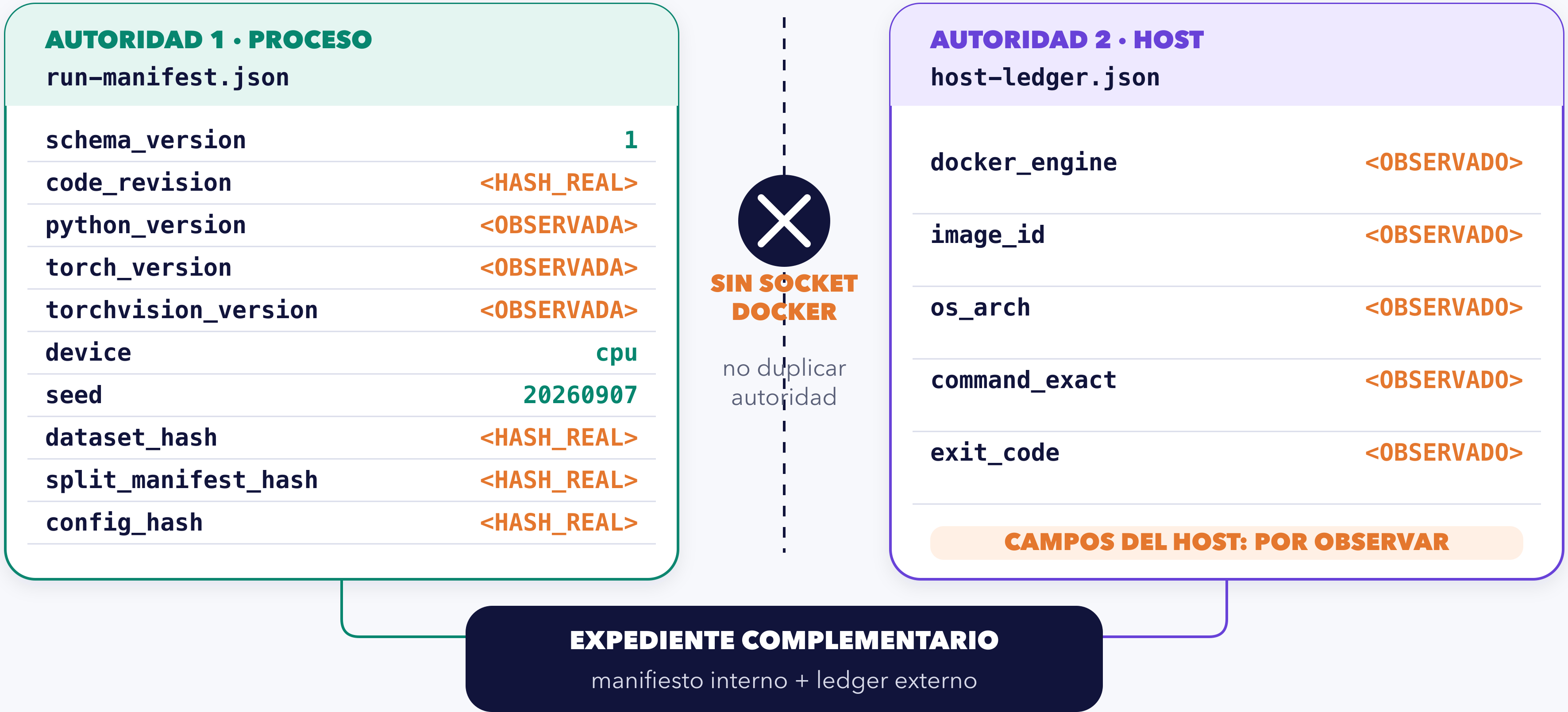
LOGS host
POR OBSERVAR

exit code host
POR OBSERVAR

5 CONTRASTES pareja
POR OBSERVAR

El proceso registra el experimento; el host completa imagen, plataforma, comando y exit code

Dos autoridades complementarias evitan que el contenedor invente metadatos de Docker.



La corrida se valida por artefactos, persistencia, entradas intactas y un fallo de escritura esperado

Después del run se inspeccionan evidencias; ninguna comprobación aparece aquí como ya observada.

FASE POST-RUN CHECKS • POR OBSERVAR

INSPECCIÓN POSITIVA • SALIDA PERSISTENTE

run-manifest.json	POR OBSERVAR
metrics.json	POR OBSERVAR
predictions.json	POR OBSERVAR
model-state.pt	POR OBSERVAR

container • image_id
exit_code

POR OBSERVAR

PRUEBA NEGATIVA ÚNICA • ENTRADA READ-ONLY

REHASH DE ENTRADAS
/data + /config SIN CAMBIO? • POR OBSERVAR

MUTACIÓN CONTROLADA
escritura en /data DEBE FALLAR • POR OBSERVAR

PERMISOS DE SALIDA
--user • por documentar

SIN --privileged

CHECKLIST POST-RUN • CERRAR SOLO CON EVIDENCIA OBSERVADA

4 ARTEFACTOS
PENDIENTE

CONTENEDOR AUSENTE
PENDIENTE

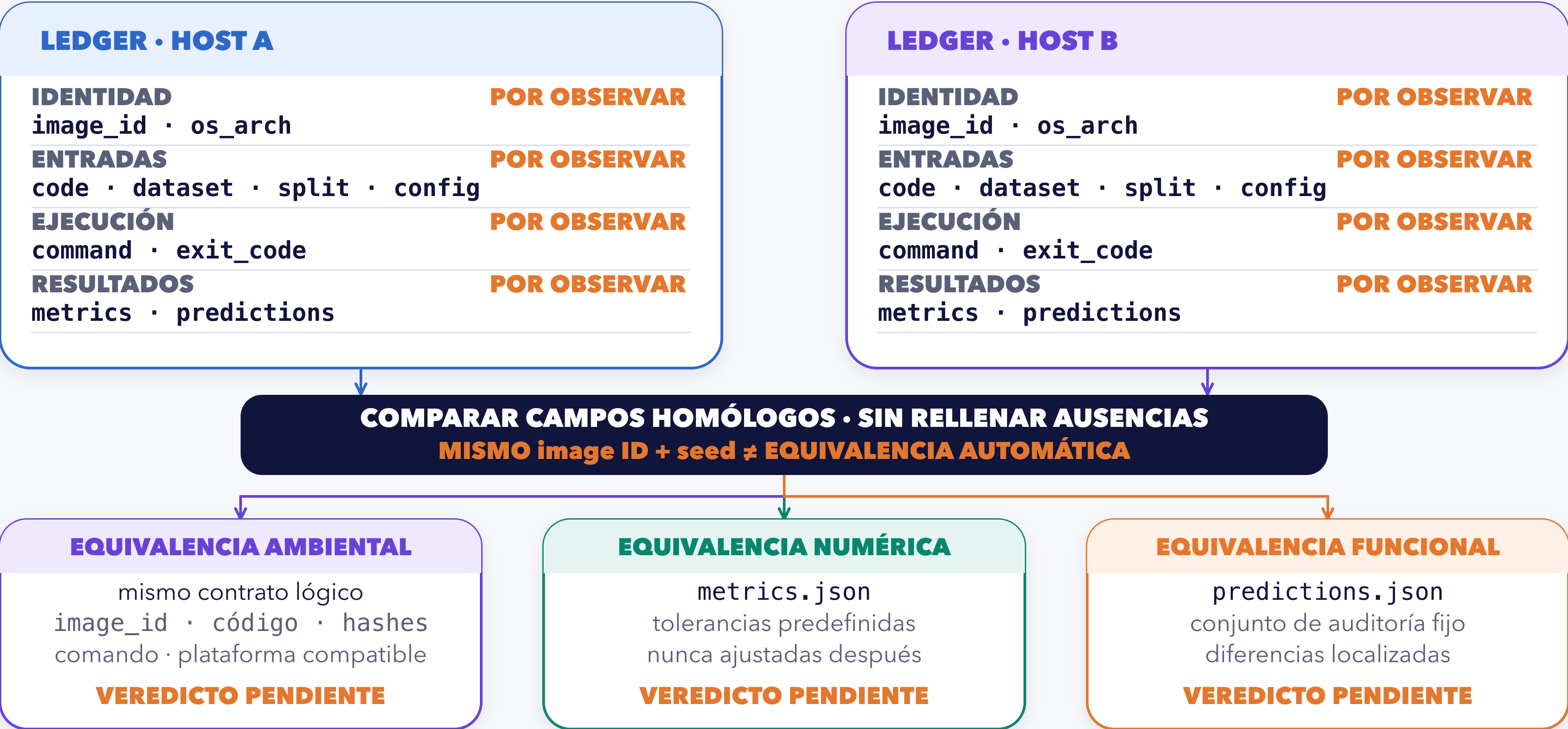
IMAGEN PRESENTE
PENDIENTE

INPUTS INTACTOS
PENDIENTE

FALLO READ-ONLY
PENDIENTE

Dos hosts se comparan por evidencia ambiental, numérica y funcional

Compartir imagen y seed no basta: cada nivel exige campos y criterios distintos.



La reproducibilidad se defiende nombrando qué se fijó, cómo se verificó y qué quedó fuera

Cierra con una afirmación que otra persona pueda auditar, no con una promesa absoluta.

1 • CONDICIÓN FIJADA

La imagen fija

MECANISMO CONCRETO

y lo verifico mediante

EVIDENCIA OBSERVABLE

2 • LÍMITE EXPLÍCITO

Docker no garantiza

LÍMITE CONCRETO

por eso registro

EVIDENCIA COMPENSATORIA

MECANISMO + EVIDENCIA + LÍMITE

AFIRMACIÓN AUDITABLE Y DELIMITADA

NO: "EL ENTORNO" • "TODO IGUAL" • "LAS DIFERENCIAS"